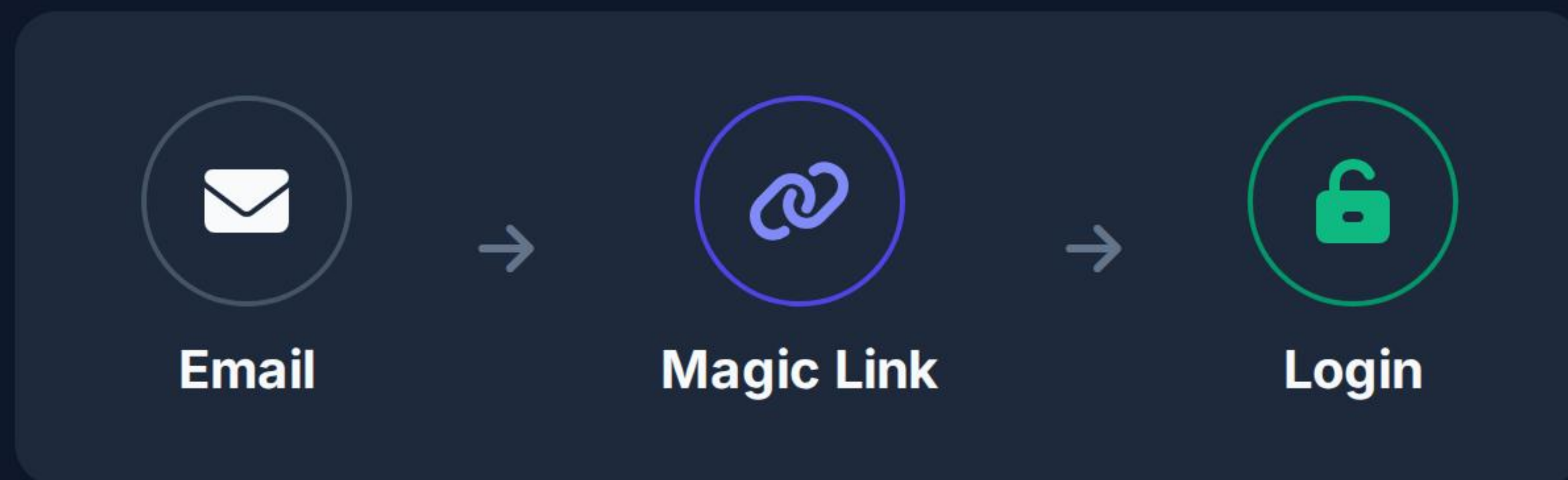


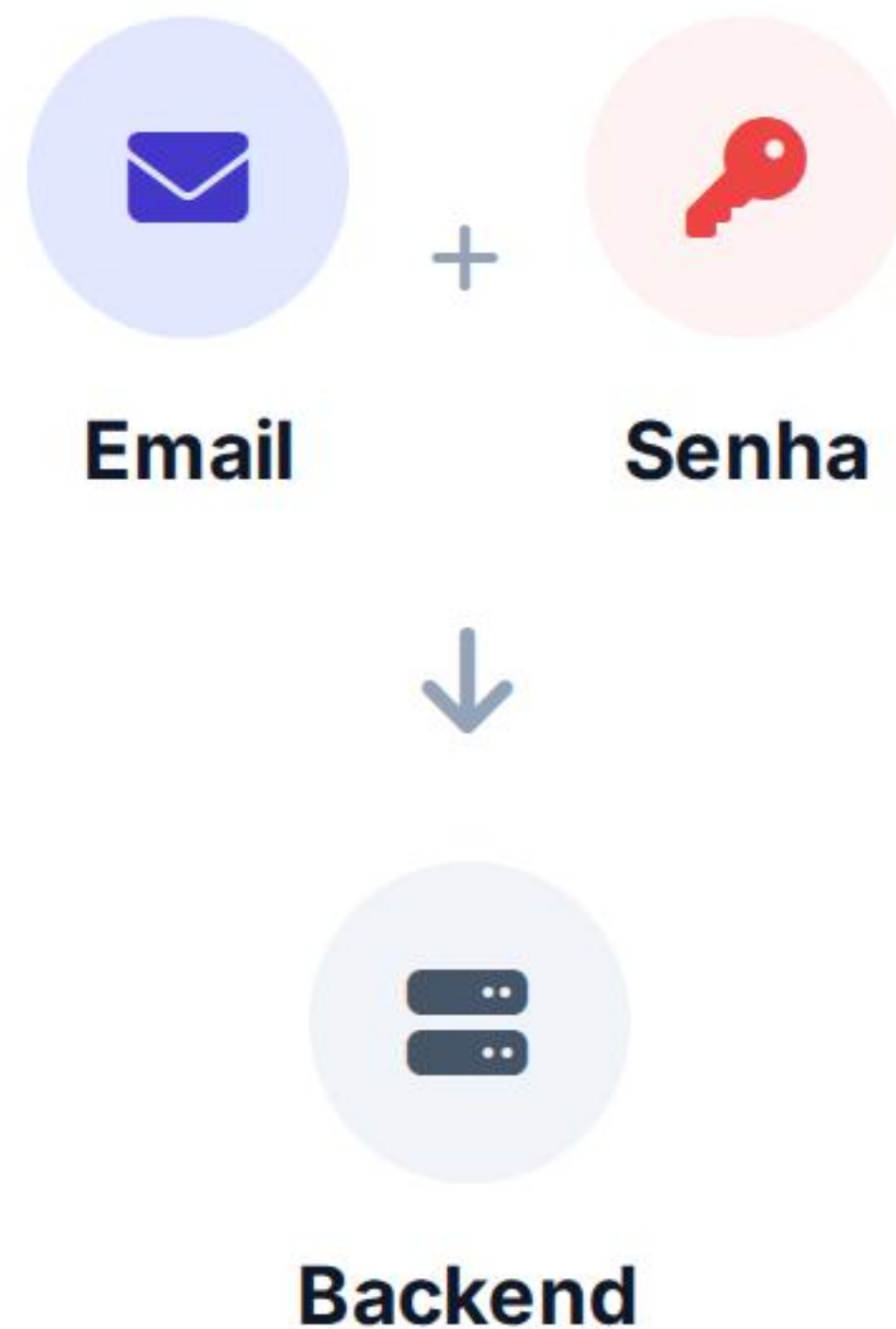
Login Mágico em Express.js e NestJS

Autenticação sem senha usando Magic Link

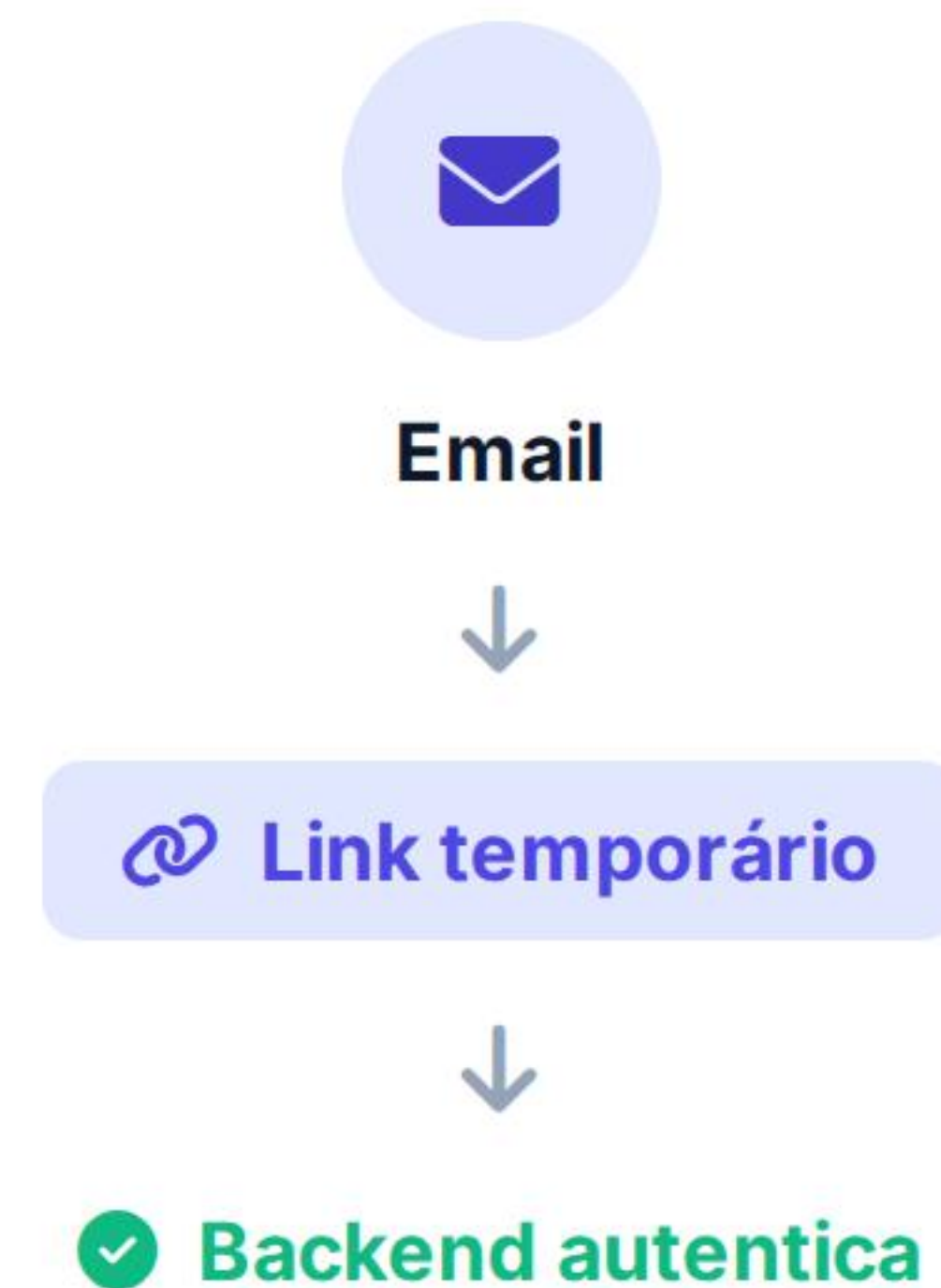


O que é Login Mágico?

Login Tradicional



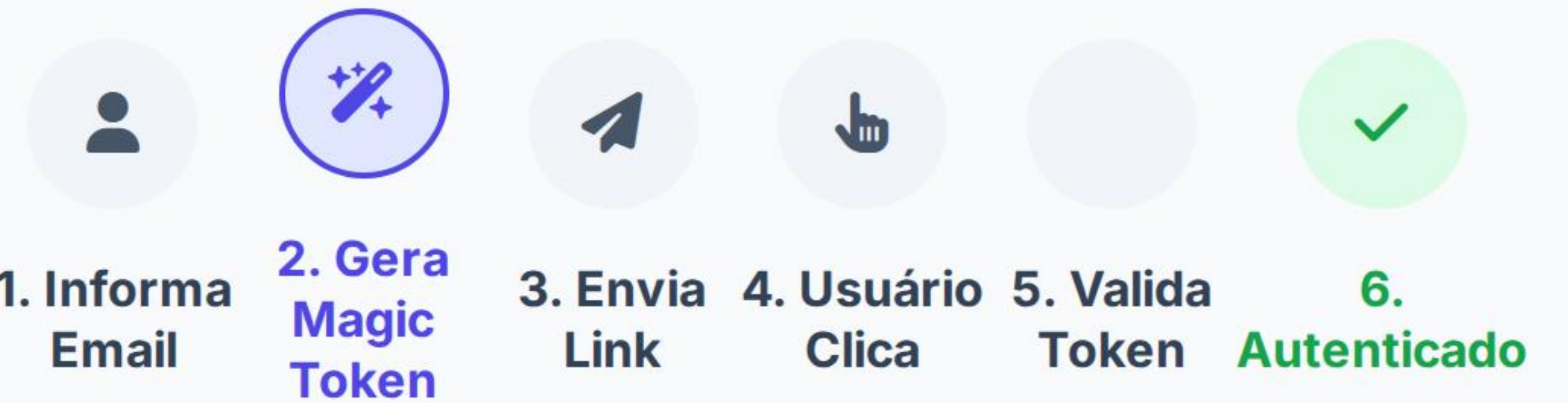
Login Mágico



Conceito: Link temporário enviado ao email para concluir o login sem senha.

🛡️ A aplicação não precisa armazenar a senha do usuário.

O Fluxo de Autenticação



Magic Token

Garante e autentica o momento inicial do login via email, validando a identidade do usuário de forma segura e sem a necessidade de senhas.

Arquitetura do Fluxo Completo

📱 CLIENTE

1. Informa email

POST /auth/request-link

4. Clica no link do email

GET /auth/verify?token= ...

6. Acessa recursos

GET /auth/profile

Header: Cookie de Sessão

🖥️ BACKEND

2. Gera Magic Token

Salva no banco (hash/expiração)

✉️ 3. Envia link por Email

5. Valida Token & Inicia Sessão

Verifica expiração, inativa Magic Token e cria sessão

7. Valida Sessão

Permite acesso à rota protegida

Magic Link com Express.js

ARQUITETURA SIMPLES

 Routes (Mapeamento)



 Controller (Middlewares)



 Service (Regra de Negócio)

1. Solicitar Link:

```
router.post(
  "/request-link",
  authController.requestLink
);
```

2. Verificar e Autenticar:

```
router.get(
  "/verify",
  authController.verifyLink
);
```

- ✓ Lógica concentrada no Service.
- ✓ Nodemailer para envio do email.
- ✓ Sessão do usuário é iniciada no GET `/verify`.

> Demonstração na Prática

```
POST /auth/request-link  
{ "email": "teste@email.com" }
```



LINK RECEBIDO NO EMAIL:

```
GET http://localhost:3000/auth/verify?token=xyz123
```

↳ Usuário clica → Token validado → Retorna { status: "sucesso" }



```
GET /auth/profile
```

Cookie: session_id=abc12345

Magic Link com NestJS

ESTRUTURA MODULAR

AuthModule

- └─ Controller (Recebe requisição)
- └─ Service (Regra de negócio)

1. Endpoint de Solicitação:

```
@Post("request-link")
requestLink(@Body() body) {
  return this.authService.requestLink(body.email);
}
```

2. Endpoint de Validação:

```
@Get("verify")
verify(@Query("token") token: string) {
  return this.authService.verifyLink(token);
}
```

NestJS: Protegendo Rotas

Utilizando a sessão estabelecida pelo Magic Link

```
@Get("profile")
@UseGuards(AuthGuard)
getProfile(@Req() req) {
  return req.user;
}
```

Depois que o Magic Link autentica o usuário, a **sessão criada** passa a ser utilizada para acessar os recursos protegidos da API.



1. Cookie de Sessão



2. AuthGuard



3. SessionStrategy (Validação)



4. Acesso Liberado (req.user)

Express vs NestJS

O fluxo não muda; a organização sim.

 **Express.js**

 **NestJS**

Routes & Middlewares

Controllers & Guards

Lógica em Services

Lógica em Services (Injeção de Dependência)

Configuração mais manual

Arquitetura estritamente modular

JavaScript (ou TS manual)

TypeScript nativo

Passport é opcional/manual

Integração estruturada via módulos (Passport)

Mesmo conceito → Implementações diferentes

Resumo

1

✉ Usuário informa
o email

2

🔗 Recebe um
Magic Link

3

✅ Link validado &
autenticado

4

🔌 Sessão iniciada
para a API

Email > Magic Link > Autenticação > Sessão Ativa > Rotas protegidas

Login sem senha, com um fluxo simples de entender e implementar.